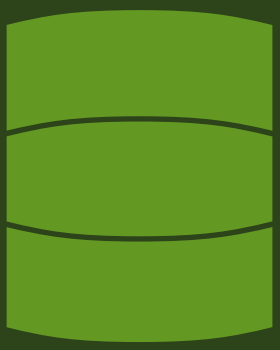

Spring Data JPA

Complete Reference Guide

Spring Boot · MySQL · Java

TABLE OF CONTENTS

All topics covered in this guide

01

The Data Access Evolution

In-Memory → JDBC → Spring JDBC → JPA → Spring Data JPA

Each stage in Java database access evolution solved a specific pain point from the previous stage. Understanding this history explains why each annotation and concept in Spring Data JPA exists.

Stage	Technology	Problem Solved	Remaining Problem
1	In-Memory (ArrayList)	No setup needed	Data lost on every restart
2	Plain JDBC	Real database persistence	30+ lines per query, manual cleanup
3	Spring JDBC	Auto connection and exception handling	Still writing SQL manually
4	JPA + Hibernate	ORM — objects map to tables, no SQL for CRUD	Complex XML/annotation configuration
5	Spring Data JPA	Zero boilerplate — extend one interface, get full CRUD	None — this is the destination

Note: Each stage becomes obvious once you have felt the pain of the previous one. Plain JDBC teaches you why Spring JDBC exists. Spring JDBC teaches you why ORM exists. ORM teaches you why Spring Data JPA exists.

02

Spring JDBC — JdbcTemplate and RowMapper

Spring handles the connection. You handle the SQL.

What is Spring JDBC?

Spring JDBC is a module that wraps plain JDBC inside the JdbcTemplate class. It automatically handles connection acquisition, statement creation, ResultSet iteration, and resource cleanup. You provide only the SQL and a RowMapper to convert each row into a Java object.

Why was it created?

Plain JDBC forces you to write try-catch-finally blocks around every query just to close the Connection, PreparedStatement, and ResultSet — even when an exception occurs. Spring JDBC eliminates all of this ceremony. It also translates all SQLExceptions into unchecked DataAccessExceptions so you are not forced to catch them everywhere.

The three core methods

- **update(sql, args...)** — for INSERT, UPDATE, DELETE. Returns rows affected.
- **query(sql, rowMapper, args...)** — for SELECT returning multiple rows as a List.
- **queryForObject(sql, rowMapper, args...)** — for SELECT returning exactly one row.

What is RowMapper?

RowMapper is a functional interface with one method: mapRow(ResultSet rs, int rowNum). Spring calls this method once for each row returned by the query. You read column values from the ResultSet and populate a Java object. Spring handles opening and closing the ResultSet.

RowMapper — converts each ResultSet row into an Employee object

```
public class EmployeeRowMapper implements RowMapper<Employee> {

    @Override
    public EmployeemapRow(ResultSet rs, int rowNum)
                          throws SQLException {
        Employeeemp=    new    Employee();
        emp.setId(rs.getInt("id"));
        emp.setName(rs.getString("name"));
        emp.setCity(rs.getString("city"));
        return emp;
    }
}
```

JdbcTemplate in a DAO

Spring injects a pre-configured JdbcTemplate bean into your DAO. You use it directly — no connection management needed.

JdbcTemplate — all JDBC boilerplate removed

```
@Repository
public class EmployeeDaoImpl implements EmployeeDao {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    // INSERT — Spring opens connection, executes, closes
    public void addEmployee(Employee emp) {
        String sql = "INSERT INTO employee(name, city) VALUES(?, ?)";
        jdbcTemplate.update(sql, emp.getName(), emp.getCity());
    }

    // SELECT ALL — Spring calls RowMapper for each row
    public List<Employee> getAllEmployees() {
        String sql = "SELECT * FROM employee";
        return jdbcTemplate.query(sql, new EmployeeRowMapper());
    }

    // SELECTONE—exactly one row expected
    public EmployeegetById(int id) {
        String sql ="SELECT * FROM employee WHERE id = ?";
        return jdbcTemplate.queryForObject(
                sql,new EmployeeRowMapper(), id);
    }

    // DELETE
    public void deleteEmployee(int id) {
        jdbcTemplate.update("DELETE FROM employee WHERE id = ?", id);
    }
}
```

Note: Spring JDBC translates all SQLExceptions to DataAccessException (unchecked). You are never forced to write try-catch around every DB call. Spring handles connection pooling through HikariCP automatically.

03 JPA Basics — @Entity, @Table, @Id, @GeneratedValue, @Column

Stop thinking in tables. Start thinking in objects.

What is JPA?

JPA (Java Persistence API) is a specification that defines how Java objects map to database tables. Hibernate is the most widely used implementation. Spring Data JPA sits on top of Hibernate and adds further simplification. The core concept is ORM — Object Relational Mapping.

The ORM mapping

- Java class → database table
- Java object (instance) → one row in the table
- Java field → one column in the table

Core annotations

Annotation	What it does
@Entity	Marks the class as a JPA entity — Hibernate creates a table for it
@Table(name="...")	Sets the table name. If omitted, class name is used as table name
@Id	Marks the primary key field. Every entity must have exactly one @Id
@GeneratedValue(IDENTITY)	DB auto-increments the primary key: 1, 2, 3...
@Column(name="...")	Maps field to a specific column name. Optional if names match
@Column(nullable=false)	Adds NOT NULL constraint to the column
@Column(updatable=false)	Column is excluded from UPDATE statements — set once only

@Entity — Java class mapped to database table

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;           // DB generates: 1, 2, 3...

    @Column(name = "name", nullable = false)
    private String name;       // NOT NULL in DB

    private String city;       // column name matches field name

    public Employee() {}       // required by JPA — do not remove
    // getters and setters...
}

// Hibernate generates this DDL automatically:
// CREATE TABLE employee (
//     id    BIGINT NOT NULL AUTO_INCREMENT,
//     name VARCHAR(255) NOT NULL,
//     city VARCHAR(255),
//     PRIMARY KEY (id)
// ) engine=InnoDB;
```

Note: The default constructor is mandatory. Hibernate uses reflection to create instances when loading data from the database. Without it, Hibernate throws an InstantiationException.

04

Spring Data JPA — JpaRepository and CRUD

Extend one interface. Get everything for free.

What is JpaRepository?

JpaRepository is an interface in Spring Data JPA. By extending it in your repository interface, Spring automatically generates all CRUD operations at startup. You write zero implementation code — Spring reads the interface and creates a working proxy.

How to use it

Declare an interface that extends JpaRepository, pass the entity type and its primary key type as type parameters, and leave the body completely empty. That is all that is needed for full CRUD.

Repository — empty interface gives you full CRUD

```
// Completely empty — Spring generates everything
public interface EmployeeRepository
        extends JpaRepository<Employee, Long> {
}
//                              ↑         ↑
//                          Entity     PK type
```

All built-in methods

Method	SQL generated	Returns
save(entity)	INSERT if new id, UPDATE if existing id	Saved entity
findById(id)	SELECT WHERE id = ?	Optional
findAll()	SELECT * FROM employee	List
deleteById(id)	DELETE WHERE id = ?	void
count()	SELECT COUNT(id) FROM employee	long
existsById(id)	SELECT count(*) > 0 WHERE id = ?	boolean
saveAll(list)	Multiple INSERT statements	List
findAll(Sort)	SELECT * ORDER BY ...	List

Service — using all built-in JpaRepository methods

```
// Service using the repository
@Service
public class EmployeeServiceImpl implements EmployeeService {

    @Autowired
    private EmployeeRepository repo;

    public Employee save(Employee e) { return repo.save(e); }

    public EmployeegetById(Long id) {
        return repo.findById(id)
                    .orElseThrow(() ->
                        new RuntimeException("Not found: " + id));
    }

    public List<Employee> getAll() { return repo.findAll(); }
    public void delete(Long id)    { repo.deleteById(id); }
    public long count()            { return repo.count(); }
    public boolean exists(Long id) { return repo.existsById(id); }
}
```

Note: findById returns Optional not Employee directly. Use .orElseThrow() to get the entity or throw an exception if it does not exist. Never use .get() directly — it throws NoSuchElementException with no useful message.

05

Derived Query Methods

Extend one interface. Get everything for free.

What are derived query methods?

Derived query methods are repository methods whose SQL is generated automatically by Spring Data JPA from the method name alone. You declare the method following a specific naming convention — Spring parses the name and constructs the correct WHERE clause. No @Query annotation needed, no SQL written anywhere.

Naming pattern

The pattern is: **findBy** + FieldName + optional Condition. Spring reads left to right, identifies field names from your entity, and builds the SQL accordingly.

Derived methods — SQL generated from method name

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Long> {

    // WHERE city = ?
    List<Employee> findByCity(String city);

    // WHERE name = ? AND city = ?
    List<Employee> findByNameAndCity(String name, String city);

    // WHERE name LIKE %keyword%
    List<Employee> findByNameContaining(String keyword);

    // WHERE salary > ?
    List<Employee> findBySalaryGreaterThan(Double amount);

    // WHERE salary BETWEEN ? AND ?
    List<Employee> findBySalaryBetween(Double min, Double max);

    // SELECT COUNT(*) WHERE city = ?
    long countByCity(String city);

    // SELECT count() > 0 WHERE name = ?
    boolean existsByName(String name);

    // DELETE WHERE city = ?
    @Transactional
    void deleteByCity(String city);

    // ORDER BY name ASC
    List<Employee> findAllByOrderByNameAsc();
}
```

Keywords Spring recognises

Keyword	SQL produced
And	WHERE a = ? AND b = ?
Or	WHERE a = ? OR b = ?
Containing	WHERE name LIKE %?%
StartingWith	WHERE name LIKE ?%
GreaterThan	WHERE salary > ?
Between	WHERE salary BETWEEN ? AND ?
OrderByNameAsc	ORDER BY name ASC
OrderByNameDesc	ORDER BY name DESC

Note: deleteByCity requires @Transactional on the method because delete operations must run inside a transaction. Spring will throw TransactionRequiredException without it.

06

Custom @Query — JPQL and Native SQL

When the method name is not enough, write your own query.

What is @Query?

@Query lets you write your own query when derived method names become too complex or unreadable. Spring Data JPA supports two query languages: JPQL and Native SQL.

JPQL vs Native SQL

- **JPQL** uses your Java entity class names and field names — not table or column names. It is database-independent.
- **Native SQL** uses actual table and column names from the database. It is tied to a specific database.

@Query — JPQL and Native SQL with @Param binding

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Long> {

    // JPQL — uses class name 'Employee', field name 'city'
    @Query("SELECT e FROM Employee e WHERE e.city = :city")
    List<Employee> getByCity(@Param("city") String city);

    // JPQL—multiple conditions
    @Query("SELECT e FROM Employee e " +
           "WHERE e.city = :city AND e.salary > :min")
    List<Employee> getByCityAndMinSalary(
            @Param("city") String city,
            @Param("min") Double min);

    // Native SQL — uses actual table name 'employee'
    @Query(value = "SELECT * FROM employee WHERE city = :city",
           nativeQuery = true)
    List<Employee> getByCityNative(@Param("city") String city);
}
```

Note: Prefer JPQL over native SQL when possible. JPQL is database-agnostic — the same query works on MySQL, PostgreSQL, and Oracle. Native SQL is specific to one database engine.

07

@Modifying — Custom UPDATE and DELETE

Tell Spring this query changes data, not just reads it.

What is @Modifying?

@Modifying must be added alongside @Query when the query modifies data. Without it, Spring assumes every @Query is a SELECT and returns data. @Modifying signals that this query changes data in the database. The return type should be int (rows affected) or void.

Why @Transactional is also required

All data-modifying operations must run inside a transaction. @Transactional on the method creates or joins a transaction. Without it, Spring throws TransactionRequiredException at runtime.

@Modifying + @Transactional — required pair for custom DML

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Long> {

    // Custom UPDATE
    @Modifying
    @Transactional
    @Query("UPDATE Employee e SET e.city = :city WHERE e.id = :id")
    intupdateCity(@Param("id") Long id,
                  @Param("city") String city);
    // returns number of rows updated

    // Custom DELETE
    @Modifying
    @Transactional
    @Query("DELETE FROM Employee e WHERE e.city = :city")
    int deleteByCity(@Param("city") String city);
    // returns number of rows deleted
}
```

Note: The return type int tells you how many rows were affected. If you try to run a @Modifying query without @Transactional, Spring throws: javax.persistence.TransactionRequiredException. Always use both annotations together.

08

@Transactional — ACID Transactions

All or nothing. Either everything saves, or nothing does.

What is @Transactional?

@Transactional wraps a service method in a database transaction. Either ALL operations inside succeed and commit to the database, or if ANY operation fails, ALL changes are rolled back — leaving the database as if nothing happened. This guarantees data integrity.

ACID properties

- **Atomicity** — all operations succeed together or none of them do.
- **Consistency** — database moves from one valid state to another valid state.
- **Isolation** — concurrent transactions do not see each other's uncommitted data.
- **Durability** — once committed, data survives system crashes and restarts.

Rollback rules

- Default — automatic rollback on RuntimeException and Error only.
- rollbackFor=Exception.class — also rollback on checked exceptions.

@Transactional — fund transfer: all or nothing

```
@Service
public class BankService {

    @Transactional
    public voidtransferFunds(Long fromId, Long toId,
                             Double amount) {
        BankAccountsender    = repo.findById(fromId).get();
        BankAccountreceiver = repo.findById(toId).get();

        if (sender.getBalance() < amount) {
            throw new RuntimeException("Insufficient balance");
            // RuntimeException → @Transactional rolls back both saves
            // Neither account is changed in the database
        }

        sender.setBalance(sender.getBalance() - amount);
        receiver.setBalance(receiver.getBalance() + amount);

        repo.save(sender);    // debit
        repo.save(receiver);  // credit
        // Both saves succeed → transaction commits atomically
    }
}
```

Note: Always annotate the service layer with @Transactional — not the repository. A single service method often calls multiple repository methods that must be atomic together. If you annotate the repository, each repo call gets its own transaction and they cannot roll back together.

09

Entity Relationships — @OneToMany and @ManyToOne

One parent, many children. One foreign key connects them all.

What is this relationship?

@OneToMany and @ManyToOne are two sides of the same relationship. One parent can have many children, and each child belongs to one parent. Example: one Customer has many Accounts. Each Account belongs to one Customer.

Owning side vs inverse side

- **@ManyToOne side** — the OWNING side. Holds the foreign key column via @JoinColumn. JPA writes to this side.
- **@OneToMany side** — the INVERSE side. Uses mappedBy referencing the field name in the owning entity. For navigation only.

@OneToMany and @ManyToOne — Customer has many Accounts

```
// CUSTOMER — one customer has many accounts
@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "customer",    // field name in Account
               cascade = CascadeType.ALL,
               fetch = FetchType.LAZY)
    privateList<Account> accounts;
}

// ACCOUNT — each account belongs to one customer
@Entity
public class Account {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Double balance;

    @ManyToOne                          // owning side
    @JoinColumn(name= "customer_id")    // FK column in account table
    private Customercustomer;
}

// Tables created:
// customer(id, name)
// account(id, balance, customer_id)  ← customer_id is the FK
```

Note: Built in Ex_018. Deleting a Customer with CascadeType.ALL automatically deletes all its Accounts. Hibernate handles the order — deletes children before the parent to respect foreign key constraints.

10

Entity Relationships — @OneToOne

One entity, one partner. No sharing allowed.

What is @OneToOne?

@OneToOne maps a relationship where each entity on one side is associated with exactly one entity on the other side. Example: each Customer has exactly one KYC record. The uniqueness is enforced by adding unique=true to the @JoinColumn — this creates a UNIQUE constraint on the foreign key column in the database, preventing two records from pointing to the same parent.

@OneToOne — Customer has exactly one KYC

```
// KYC — owning side, holds the foreign key
@Entity
public class Kyc {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String documentNumber;
    private boolean verified;

    @OneToOne
    @JoinColumn(name = "customer_id", unique = true)
    // unique = true → only one KYC per customer enforced in DB
    private Customer customer;
}

// CUSTOMER — inverse side
@Entity
public class Customer {
    @OneToOne(mappedBy = "customer",
              cascade = CascadeType.ALL,
              fetch = FetchType.LAZY)
    private Kyc kyc;
}

// Tables:
// kyc(id, document_number, verified, customer_id UNIQUE)
//UNIQUE constraint
```

Note: The only structural difference between @ManyToOne and @OneToOne is the unique=true on @JoinColumn. Without it the relationship behaves as @ManyToOne. With it, the database enforces the one-to-one constraint.

11

Entity Relationships — @ManyToMany

Many to many — one join table handles it all.

What is @ManyToMany?

@ManyToMany maps a relationship where many records on one side can be associated with many records on the other side. Example: one Customer can have many Offers, and one Offer can be shared by many Customers. Because no single table can hold a foreign key for both sides, a separate join table is created.

Why NOT CascadeType.ALL on @ManyToMany?

Offers are shared entities — multiple customers can hold the same Offer. If you use CascadeType.ALL, deleting a Customer would try to delete the shared Offer. But if another Customer still holds that Offer, the database throws a foreign key constraint violation. Use PERSIST and MERGE only.

@ManyToMany — Customer shares Offers via join table

```
// CUSTOMER — owning side (defines the join table)
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
// NOT CascadeType.ALL — deleting Customer must NOT delete Offers
@JoinTable(
    name = "customer_offer",
    joinColumns        = @JoinColumn(name = "customer_id"),
    inverseJoinColumns = @JoinColumn(name = "offer_id")
)
private List<Offer> offers = new ArrayList<>();

// OFFER — inverse side
@ManyToMany(mappedBy = "offers")
private List<Customer> customers = new ArrayList<>();

// Tables created:
// offer(id, title, discount)
// customer_offer(customer_id, offer_id)   ← join table
```

Lazy loading fix

With FetchType.LAZY (the default), accessing offers outside a transaction causes LazyInitializationException. The session is already closed by the time forEach tries to load the collection. Wrap in a @Transactional method and return a new ArrayList to force loading while the session is still open.

Fix for LazyInitializationException on @ManyToMany

```
// WRONG — session closes before forEach runs
public List<Offer> getOffers(Long customerId) {
    Customer c = repo.findById(customerId).get();
    return c.getOffers();      // proxy — not loaded yet
}
// calling .forEach() on result → LazyInitializationException
// CORRECT — session stays open, new ArrayList forces loading
@Transactional
public List<Offer> getOffers(Long customerId) {

    Customer c = repo.findById(customerId).get();
    return new ArrayList<>(c.getOffers()); // loaded inside session
}
```

Note: Verified : "Zero Fee Transfer" offer was stored once in the offer table. Both Varun and Kiran had entries in customer_offer pointing to it. Deleting Kiran removed only his customer_offer rows — the offer itself stayed.

12

CascadeType and FetchType

Control what spreads and control what loads.

CascadeType — which operations propagate to children

CascadeType controls which database operations on a parent entity automatically apply to its related entities. Without cascading, you must manually save or delete each related entity.

CascadeType	What cascades to children
PERSIST	Saving parent → children also saved automatically
MERGE	Updating parent → children also updated
REMOVE	Deleting parent → children also deleted
REFRESH	Refreshing parent from DB → children also refreshed
ALL	All of the above applied together

FetchType — when does related data load?

FetchType controls WHEN Hibernate loads the associated entities from the database.

FetchType	When loaded	Default for
LAZY	Only when you access the field — a separate SQL runs at that point	@OneToMany, @ManyToMany
EAGER	Immediately with the parent — included in the same SQL JOIN	@ManyToOne, @OneToOne

CascadeType selection — owned vs shared entities

```
// Use ALL for owned entities — Customer owns its Accounts
@OneToMany(mappedBy = "customer",
           cascade = CascadeType.ALL,     // save and delete cascade down
           fetch = FetchType.LAZY)        // accounts load only when accessed
private List<Account> accounts;

// Use PERSIST+MERGE for shared entities — Offers are shared
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
// NOT ALL — deleting Customer must NOT delete shared Offers
private List<Offer> offers;
```

Note: Prefer FetchType.LAZY everywhere unless you always need the related data. EAGER causes JOIN queries every time the parent loads — even when you do not need the children. This becomes a performance problem with large datasets.

13 Pagination and Sorting

Stop loading everything. Fetch only what you need.

What is pagination?

Pagination fetches a fixed number of records at a time instead of loading everything at once. Without pagination, querying 100,000 employees loads all 100,000 into memory. Pagination limits each query to a manageable page of results using SQL LIMIT and OFFSET.

Key classes

- **PageRequest.of(page, size)** — creates a Pageable with page number (0-indexed) and page size.
- **PageRequest.of(page, size, Sort.by("field"))** — adds sorting.
- **Page** — the result. Contains the records for this page and metadata about all pages.

OFFSET formula

Hibernate converts page and size to SQL: OFFSET = page × size. Page 0 size 3 → OFFSET 0. Page 1 size 3 → OFFSET 3. Page 2 size 3 → OFFSET 6.

Pagination — repository and service methods

```
// Repository — add custom paginated method
public interface EmployeeRepository
        extends JpaRepository<Employee, Long> {
    Page<Employee> findByCity(String city, Pageable pageable);
}

// Service — three pagination patterns
public Page<Employee> getAllPaged(int page, int size) {
    return repo.findAll(PageRequest.of(page, size));
}
// SQL: SELECT * FROM employee LIMIT 3 OFFSET 0
// SQL: SELECT count(id) FROM employee ← runs automatically for totals

public Page<Employee> getSortedByName(int page, int size) {
    return repo.findAll(
            PageRequest.of(page, size, Sort.by("name")));
}
// SQL: SELECT * FROM employee ORDER BY name ASC LIMIT ? OFFSET ?

public Page<Employee> getByCity(String city, int page, int size) {
    return repo.findByCity(city, PageRequest.of(page, size));
}
// SQL: SELECT * FROM employee WHERE city=? LIMIT ? OFFSET ?
```

Page object methods

Page — all available metadata methods

```
Page<Employee> page = service.getAllPaged(0, 3);

page.getContent();          // List<Employee> — records on this page
page.getTotalElements();    // total records matching query in DB
page.getTotalPages();       // ceil(totalElements / size)
page.getNumber();           // current page number (0-indexed)
page.isFirst();             // true on page 0
page.isLast();              // true when no more pages
page.hasNext();             // is there a next page?
page.hasPrevious();         // is there a previous page?
```

Note: Hibernate runs TWO queries for every Page request: one for the data (LIMIT/OFFSET) and one for the total count (SELECT COUNT). The count query is needed to calculate totalPages and totalElements. Both run automatically.

14

JPA Auditing — @CreatedDate and @LastModifiedDate

Spring sets the timestamps. You never forget to.

What is JPA Auditing?

JPA Auditing automatically populates timestamp fields when entities are inserted or updated. Without it, you must manually write createdAt = LocalDateTime.now() in every save call — and you risk forgetting. With auditing enabled, Spring handles all timestamps with zero manual code.

What you need

- **@EnableJpaAuditing** — added to the main application class once. Enables auditing globally.
- **@EntityListeners(AuditingEntityListener.class)** — added to the entity class to register it for audit events.
- **@CreatedDate** — set by Spring once on INSERT. Use with @Column(updatable=false) to lock it.
- **@LastModifiedDate** — updated by Spring every time save() is called.

@EnableJpaAuditing — automatic timestamps

```
// Enable once in your main class
@SpringBootApplication
@EnableJpaAuditing
public class Application { ... }

// Entity — add listener and two auditing fields
@Entity
@EntityListeners(AuditingEntityListener.class)
public class Employee {

    @CreatedDate
    @Column(updatable = false)   // never included in UPDATE
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;
}

// What happens automatically:
//  INSERT→ createdAt = now,  updatedAt = now
// UPDATE → createdAt unchanged,  updatedAt = now
```

Console proof — createdAt never changes after INSERT

```
// After createEmployee("Varun", "Hyderabad"):
// createdAt = 2026-05-02T17:01:46.085820
// updatedAt = 2026-05-02T17:01:46.085820

// After updateEmployee(id, "Delhi"):
// UPDATE employee SET city=?, name=?, updated_at=? WHERE id=?
// Note: created_at is NOT in the UPDATE statement

// createdAt = 2026-05-02T17:01:46.085820 ← unchanged
// updatedAt = 2026-05-02T17:01:46.113509  ← changed
```

Note: @Column(updatable=false) on createdAt is critical. Even if someone passes a different createdAt value to save(), Hibernate excludes that column from the UPDATE statement entirely. The created timestamp is permanently locked after the first INSERT.

15

Entity Lifecycle Callbacks

Hook into every database moment before and after it happens.

What are lifecycle callbacks?

Lifecycle callbacks are methods you define inside your @Entity class that JPA calls automatically at specific database events. They give you hooks to run custom logic at exactly the right moment — before insert, after insert, before update, after update, before delete, after delete, and after a select loads an entity.

All seven callbacks

Annotation	When it fires	id available?	Common use
@PrePersist	Before INSERT	No — still null	Set default values, timestamps
@PostPersist	After INSERT	Yes — DB assigned	Log with id, send email
@PreUpdate	Before UPDATE	Yes	Update timestamp, validate
@PostUpdate	After UPDATE	Yes	Send notification, log change
@PreRemove	Before DELETE	Yes	Archive data before it is gone
@PostRemove	After DELETE	Yes	Confirm deletion, cleanup
@PostLoad	After SELECT	Yes	Decrypt fields, log access

BankAccount — key lifecycle callbacks from Ex_022

```
@Entity
public class BankAccount {

    @PrePersist
    public void beforeCreate() {
        this.status    = "ACTIVE";       // set default — not passed by caller
        this.createdAt  = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    @PostPersist
    public void afterCreate() {
        // id is available here — DB has assigned it
        log.info("Account created id={} status={}", id, status);
    }

    @PreUpdate
    public void beforeUpdate() {
        this.updatedAt = LocalDateTime.now(); // auto-update timestamp
    }

    @PreRemove
    public void beforeDelete() {
        log.info("Deleting account: {} id={}", accountHolder, id);
    }

    @PostLoad
    public void afterLoad() {
        log.info("Loaded: {} balance={}", accountHolder, balance);
    }
}
```

Note: In @PrePersist, the id field is still null because the INSERT has not happened yet. The DB assigns the id during INSERT. Use @PostPersist when you need the generated id — for example, to log it or send a notification that includes the account id.

16

Projections — Fetch Specific Columns

Why fetch ten columns when you only need two.

What are projections?

Projections allow you to fetch only specific columns instead of loading the entire entity. When you call findAll(), Hibernate selects ALL columns — even ones you do not need. With projections, you define exactly which fields you want and Hibernate adjusts the SELECT statement accordingly.

Why use projections?

- **Performance** — fewer columns means less data transferred from the database.
- **Security** — sensitive fields like salary or password are excluded unless explicitly requested.
- **API design** — return exactly what the endpoint needs, not the full entity.

How interface-based projections work

You define a Java interface with getter methods. Each getter name determines which column is fetched — getName() fetches the name column. Spring creates a proxy at runtime that implements the interface. No implementation code needed.

Projection interfaces and repository methods

```
// Define interfaces — each fetches different columns
public interface NameCityProjection {
    String  getName();  // fetches name column
    String getCity();   // fetches city column
}

public interface NameSalaryProjection {
    String getName();
    Double getSalary(); // fetches salary column
}

// Repository — return type drives which columns appear in SQL
public interface EmployeeRepository
        extends JpaRepository<Employee, Long> {

    List<NameCityProjection> findAllNameCityBy();
    // SQL: SELECT name, city FROM employee

    List<NameSalaryProjection> findByDepartment(String dept);
    // SQL: SELECT name, salary FROM employee WHERE department=?
}
```

Reading projection results — getter methods only, not toString()

```
// Accessing projection — use getter methods
service.getNameAndCity().forEach(p ->
    System.out.println(p.getName() + " | " + p.getCity())
);

// Output from Ex_023:
// Varun | Hyderabad
// Kiran | Banglore
// Sneha | Hyderabad

// Full entity query fetches:
// SELECT id, name, city, department, salary FROM employee

// Projection query fetches only:
// SELECT name, city FROM employee
```

Note: You cannot call toString() on a projection — it returns a proxy class reference, not your field values. Always access data through the defined getter methods. If you call a getter that is not defined in the interface, you get null.

SUMMARY

Spring Data JPA — All Topics at a Glance

01	Data Access Evolution	*In-Memory → JDBC → Spring JDBC → JPA → Spring Data JPA*
02	Spring JDBC	*JdbcTemplate, RowMapper, update(), query(), queryForObject()*
03	JPA Basics	*@Entity, @Table, @Id, @GeneratedValue, @Column — ORM mapping*
04	JpaRepository	*save, findById, findAll, deleteById, count, existsById*
05	Derived Query Methods	*findByCity, findByNameAndCity, countBy, existsBy, deleteBy*
06	Custom @Query	*JPQL with class names, Native SQL with table names, @Param*
07	@Modifying	*Custom UPDATE and DELETE — always with @Transactional*
08	@Transactional	*ACID, automatic rollback on RuntimeException, fund transfer*
09	@OneToMany / @ManyToOne	*Customer↦ Account → Transaction, @JoinColumn, mappedBy*
10	@OneToOne	*Customer↦ KYC, unique=true on @JoinColumn*
11	@ManyToMany	*Customer↔ Offer, @JoinTable, PERSIST + MERGE cascade only*
12	CascadeType and FetchType	*ALL / PERSIST / MERGE, LAZY vs EAGER loading*
13	Pagination and Sorting	*PageRequest.of(), Sort.by(), Page metadata*
14	JPA Auditing	*@EnableJpaAuditing, @CreatedDate, @LastModifiedDate*
15	Entity Lifecycle	*@PrePersist, @PostPersist, @PreUpdate, @PostUpdate, @PreRemove, @PostLoad*
16	Projections	*Interface-based, SELECT only needed columns, filter + projection*